

Report - Linux Live Response Digital Forensics Collector

By: Han Nguyen, Ninel Benitez, London Morris

1. Introduction (5 points)

There is a lack of immediate forensic visibility during incidents. When a system is compromised, responders must quickly answer questions such as "Who is logged in?", "What processes are running?", and "Has persistence been established?" Manually running commands, however, is time-consuming, error-prone, and difficult to document consistently.

This tool streamlines the process with exceptional speed and accuracy, significantly reducing the risk of contamination. It addresses the challenges of inconsistent and potentially unsafe forensic evidence collection on live Linux systems by delivering a standardized, defensible live-response solution that reliably preserves critical system and persistence artifacts for thorough investigation.

The purpose of this project was to develop a Linux-based live response tool that automates the collection of forensic artifacts while preserving system integrity. During digital forensics, critical system information often needs to be gathered for data to be compromised or deleted.

This is why it is crucial to be able to efficiently collect volatile and non-volatile data from a live system in a repeatable and structured way. The live response tool focuses on identifying potential security threats and persistence mechanisms that attackers can use to access a compromised system. By automating artifacts and organizing outputs into a structured directory, the tool simplifies and accelerates the early stages of incident response and digital forensic investigations.

2. Technical Implementation:

This Linux Live Response tool was implemented using bash scripting on a Linux environment. Through the usage of bash scripting, it allowed for a direct interaction with the operating system and also allowed for access to administrative tools and already built-in forensics.

System Architecture:

Our Linux-based live response tool consisted of multiple scripts that collected various types of system data:

- **collector.sh**
 - Main script that executes all modules
 - Creates timestamped output directories
- **System_info.sh**
- Collects system details using:
 - `uname`
 - `hostnamectl`
 - `timedatectl`
 - `lscpu`
 - `Free`
- **users_auth.sh**
- Gathers user and authentication data:
 - `who`
 - `last`
 - `/etc/passwd`
 - Bash history files

processes.sh

- Captures running processes and services:
 - `ps aux`
 - `lsmod`
 - `systemctl list-units`

network.sh

- Collects network-related information:
 - `ss -tulnp`
 - `ip addr`
 - `ip route`

logs.sh

- Extracts system logs:
 - /var/log/auth.log
 - /var/log/syslog
 - journalctl

persistence.sh

- Detects persistence mechanisms:
 - Cron jobs (user and system)
 - Systemd services and timers
 - Startup scripts
 - SUID/SGID binaries

Techniques:

Many techniques were used and mainly to create a structured output organization, as well to automate repetitive tasks, preserve forensic integrity through data collection and the usage of a command line forensic analysis. All these tools were used to be able to replicate a real-world incident response that created a reliable, fast and low overhead access to system data.

This tool employs a modular code structure, with an emphasis on clean code, scalability, and maintainability. The collection logic is separated into independent modules, so new forensic artifacts can be targeted by simply adding a new script, without risk of integrity of the core execution engine.

Results:

The tool was able to generate a structured output folder containing categorized forensic data that is crucial. Through this structured approach it helps investigators be able to locate and analyze relevant data. The tool improves efficiency, supports a more streamlined investigative workflow. Tools ensure that data is easier to be viewed and not overlooked during the initial phase.

Subfolders are structured by:

- logs/
- persistence/
- processes/
- system/
- users/

Key Outcomes:

- Organized and structured data

- Direct identification of any potential persistence techniques
- Gives a collection of system artifacts within seconds

3. Results (5 points)

The results of the live response tool was successful in capturing a snapshot of the system's state without altering critical forensic evidence. We used a modular approach, which ensured that even if one collection script failed, others would continue to populate the timestamped data directory.

First, our tool was able to effectively log all active sessions and user account information by capturing `/etc/passwd` and bash histories. We were able to create an immediate trail of user activity and potential unauthorized account creation.

Secondly, the persistence module in our tool was able to successfully map out all scheduled tasks. Capturing entries from `/etc/crontab`, `/etc/cron.d/`, and individual user crontabs allowed for quick identification of suspicious scripts that were scheduled to run at boot or a recurring basis.

Finally, our tool was able to clearly correlate open network ports via `ss` with their corresponding PIDs via `ps`, allowing investigators to see exactly which process was communicating with an external IP address in real-time.

Cron jobs:

```

File Actions Edit View Help
-rwxr-xr-x 1 root root 123 Sep 10 2024 dpkg
lrwxrwxrwx 1 root root 37 Apr 21 13:50 google-chrome → /opt/google/chrome/cron/google-chrome
-rwxr-xr-x 1 root root 377 Jul 14 2024 logrotate
-rwxr-xr-x 1 root root 1395 Aug 29 2024 man-db
-rw-r--r-- 1 root root 102 Mar 26 2024 .placeholder
-rwxr-xr-x 1 root root 652 Dec 7 2020 plocate
-rwxr-xr-x 1 root root 526 Jan 15 2024 sysstat

/etc/cron.hourly:
total 20
drwxr-xr-x 2 root root 4096 Nov 30 2024 .
drwxr-xr-x 192 root root 12288 May 7 12:08 ..
-rw-r--r-- 1 root root 102 Mar 26 2024 .placeholder

/etc/cron.monthly:
total 20
drwxr-xr-x 2 root root 4096 Nov 30 2024 .
drwxr-xr-x 192 root root 12288 May 7 12:08 ..
-rw-r--r-- 1 root root 102 Mar 26 2024 .placeholder

/etc/cron.weekly:
total 24
drwxr-xr-x 2 root root 4096 Nov 30 2024 .
drwxr-xr-x 192 root root 12288 May 7 12:08 ..

```

Systemd:

```

File Actions Edit View Help
capsule@.service root root 653 Dec 7 2024 static -
colord.service root root 528 Jan 15 2024 static -
configure-printer@.service static -
console-getty.service disabled disabled
console-setup.service enabled enabled
container-getty@.service 4896 Nov 30 2024 static -
containerd.service root 12188 May 7 12:08 disabled disabled
cron.service root root 102 Mar 26 2024 enabled enabled
cryptdisks-early.service masked disabled
cryptdisks.service masked disabled
dbus-org.freedesktop.hostname1.service alias -
dbus-org.freedesktop.locale1.service 2024 alias -
dbus-org.freedesktop.login1.service 7 12:08 alias -
dbus-org.freedesktop.ModemManager1.service alias older -
dbus-org.freedesktop.nm-dispatcher.service alias -
dbus-org.freedesktop.timedate1.service alias -
dbus.service static -
debug-shell.service root 4896 Nov 30 2024 disabled disabled
display-manager.service 12188 May 7 12:08 alias -
docker.service enabled enabled
dpkg-db-backup.service static -
e2scrub@.service static -
e2scrub_all.service static -
e2scrub_fail@.service static -

```

Processes:

```

File Actions Edit View Help
root 22 0.0 0.0 0 0 ? S 12:07 0:00 [cpuhp/0]
root 23 0.0 0.0 0 0 ? S 12:07 0:00 [cpuhp/1]
root 24 0.0 0.0 0 0 ? S 12:07 0:00 [idle_inject/1]
root 25 0.0 0.0 0 0 ? S 12:07 0:00 [migration/1]
root 26 0.0 0.0 0 0 ? S 12:07 0:02 [ksoftirqd/1]
root 30 0.0 0.0 0 0 ? S 12:07 0:00 [kworker/u10:0-events_unbound]
root 33 0.0 0.0 0 0 ? S 12:07 0:00 [kdevtmpfs]
root 34 0.0 0.0 0 0 ? I< 12:07 0:00 [kworker/R-inet_frag_wq]
root 35 0.0 0.0 0 0 ? S 12:07 0:00 [kauditd]
root 36 0.0 0.0 0 0 ? S 12:07 0:00 [khungtaskd]
root 37 0.0 0.0 0 0 ? S 12:07 0:00 [oom_reaper]
root 39 0.0 0.0 0 0 ? I< 12:07 0:00 [kworker/R-writeback]
root 40 0.6 0.0 0 0 ? S 12:07 0:28 [kcompactd0]
root 41 0.0 0.0 0 0 ? S 12:07 0:00 [ksmd]
root 42 0.1 0.0 0 0 ? S 12:07 0:06 [khugepaged]
root 43 0.0 0.0 0 0 ? I< 12:07 0:00 [kworker/R-kintegrityd]
root 44 0.0 0.0 0 0 ? I< 12:07 0:00 [kworker/R-kblockd]
root 45 0.0 0.0 0 0 ? I< 12:07 0:00 [kworker/R-blkcg_punt_bio]
root 46 0.0 0.0 0 0 ? S 12:07 0:00 [irq/9-acpi]
root 48 0.0 0.0 0 0 ? I< 12:07 0:00 [kworker/R-tpm_dev_wq]
root 49 0.0 0.0 0 0 ? I< 12:07 0:00 [kworker/R-edac-poller]
root 50 0.0 0.0 0 0 ? I< 12:07 0:00 [kworker/R-devfreq_wq]
root 53 0.7 0.0 0 0 ? S 12:07 0:30 [kswapd0]
root 57 0.0 0.0 0 0 ? I 12:07 0:00 [kworker/u10:3-events_unbound]

```

Network Interfaces:

```
Shell No. 1
File Actions Edit View Help
[NETWORK INTERFACES]
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:6e:13:6e brd ff:ff:ff:ff:ff:ff
   inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
       valid_lft 82141sec preferred_lft 82141sec
   inet6 fd00::483c:f169:1579:8c9a/64 scope global dynamic noprefixroute
       valid_lft 86203sec preferred_lft 14203sec
   inet6 fe80::85a9:989f:7897:afff/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: br-9c7ac5af2bc6: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
   link/ether 56:5e:88:21:c8:d9 brd ff:ff:ff:ff:ff:ff
   inet 172.18.0.1/16 brd 172.18.255.255 scope global br-9c7ac5af2bc6
       valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
   link/ether b6:8f:00:ec:7b:dc brd ff:ff:ff:ff:ff:ff
   inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
~
~
```

AI Integration and Analysis:

```
Shell No. 1
File Actions Edit View Help
Based on the forensic artifacts provided, here is the technical analysis of the system state.

1. Executive Summary
The forensic data represents a live-response capture from a Linux-based virtual machine (x86_64). Initial analysis of the process environment and systemd unit configurations reveals a standard guest OS profile (likely Debian/Kali-based) running within a VirtualBox hypervisor. While the majority of services are in a standard state, there are specific disabled security components and a high volume of systemd units that warrant a closer look for potential persistence or evasion.

2. Suspicious Findings
Disabled Security/Network Services: Several services that are often used for secure communication or auditing are explicitly disabled (e.g., wpa_supplicant, vpnc, winbind). While common in lab environments, in a production incident response, this could indicate an effort to limit external monitoring or bypass centralized authentication.

Shadow Units: There is a large volume of unit files (282). While many are masked or static, several generated units (like systemd-getty-generator) should be cross-referenced against known-good baselines to ensure no malicious redirection of system tty or login processes.

3. Persistence Findings
VirtualBox Guest Integration: The service virtualbox-guest-utils.service is enabled. In some scenarios, vulnerabilities in guest additions can be leveraged for VM escape or shared folder manipulation.

Timer Units: There are multiple timer units active (e.g., apt-daily.timer, logrotate.timer). These are common locations for attackers to hide scheduled execution of malicious scripts.

Dormant Unit Files: A high number of unit files are in an enabled-runtime or disabled state. These represent "pre-staged" persistence locations where an attacker could easily enable a backdoor with a single systemctl command.

4. Network Findings
Current State: NetworkManager.service and systemd-networkd.service are present.

Anomaly: systemd-resolved.service is enabled, which manages local DNS. During an incident, this should be checked for "DNS Hijacking" if the system was observed communicating with suspicious C2 (Command and Control) domains.

Missing Artifacts: The current bundle lacks netstat or ss output. Without these active socket connections, I cannot confirm current outbound C2 activity.

5. Risk Assessment
Severity: Low-Medium (Based on current data).

Impact: The system appears to be a specialized tool/scanner environment. The risk lies in the potential for data exfiltration if the "Robort" integration or similar custom tools are compromised, as they may hold credentials or vulnerability data.

23,77 Top
```


4. Lessons Learned & Conclusion (10 points)

This project made us learn many lessons: some of which were technical, and others about the high-level professional forensic mindset. One of the primary lessons we learned was the challenge of data volatility. We realized that capturing data like network connections and running processes must happen in a specific order; for example, these must happen before file-system logs, to ensure that evidence isn't lost if the system is shut down, or before an attacker can clear their tracks. The order of operations is just as important as the data being collected, because every command executed has the potential to alter the system state.

Secondly, permission constraints was an issue we faced, and learned from. During development of the tool, it became clear that while many artifacts are readable by a standard user, critical forensics like `/var/log/auth.log` or SUID scans require root privileges. We had to implement checks within `main.sh` to ensure the tool was run with necessary permissions.

Finally, we learned about standardization. Inconsistent output formatting makes analysis difficult, which led to our implementation of the structured directory system, which taught us the importance of making data machine readable for future integration with SIEM or automated analysis tools.

In conclusion, this project successfully produced a functional Linux Live Response Digital Forensics Collector. By automating the collection of both volatile and non-volatile data, our tool mitigates the human error associated with manual command entry and ensures a repeatable, defensible forensic process.

The inclusion of a dedicated persistence detection module helped address a critical gap in rapid incident response, allowing investigators to identify attacker footprints, such as cron jobs and systemd timer, within seconds.

Ultimately, we learned that bash-based automation is a powerful asset in preserving the chain of custody and maintaining forensic integrity during the high-pressure early stages of an incident.